FINAL PROJECT PROPOSAL

Saint Cy's Hospital Management Website

(Web Application)

2/22/26

PREPARED FOR

SE/COM S 3190 - Construction of User Interfaces
Iowa State University
Department of Computer Science

PREPARED BY

Devank Uppal (duppal@iastate.edu)
Neal Sharma (nea1@iastate.edu)

Contents

1 Introduction **2**

2 Previous Work (COM S 3090 System) **2**

3 Problem Statement and Purpose **3**

4 Goals and Objectives **3**

5 Project Description and System Design **4**

6 Component-Based Development **4**

6.1 Wireframes 5

6.1.1 Home Page / Dashboard 5

6.1.2 Appointments Page 6

7 Feature Ownership and System Responsibilities **6**

8 API Design (High-Level) **7**

9 Database Design **7**

10 Technical Stack and Resources **8**

11 GitLab Milestones and Planning **8**

12 Error Handling and Security **9**

13 AI Feature (Optional Enhancement) **9**

14 Final Comments **9**

1 Introduction

In COM S 3090, we built a backend system for a hospital management application. In this course, we first created a working website using HTML, CSS, and JavaScript to represent that system.

Now, we are rebuilding that website using React and connecting it to a database. This allows the application to store and retrieve real data instead of using static content. Our goal is to combine a clean user interface with working backend functionality.

2 Previous Work (COM S 3090 System)

In COM S 3090, we built a full-stack hospital management system that supports multiple user roles including patients, doctors, pharmacists, and administrators. The system included backend services and a mobile interface that handled real-time data, communication, and healthcare workflows.

This project serves as the foundation for our current web application, where we are rebuilding these features using React and integrating them into a structured web-based system.

Authentication and User Roles: The system supported account creation, login, and role-based access control. Different users (patients, doctors, admins, pharmacists) had access to different features. This directly maps to our Login, Admin Panel, and Doctor Portal pages in the web application.

Patient Dashboard: Users were able to view notifications, account details, and system updates after logging in. In this project, this will be implemented as the Patient Dashboard page showing appointments, prescriptions, and alerts.

Appointments System: Patients could schedule appointments using a calendar interface, and doctors could confirm or manage those appointments. In our web application, this will be implemented in the Appointments page, where users can create, update, and manage appointment data through API calls.

Prescription and Pharmacy Workflow: Doctors could create prescriptions, which patients could then access through the pharmacy system. Pharmacists were able to manage orders by accepting, rejecting, and completing them. This functionality will be split across the Pharmacy page and Doctor Portal in our web application.

Messaging System: The system included real-time messaging between patients and doctors. Messages supported live updates and notifications. In the web application, this will be implemented as a messaging feature within the dashboard to support communication between users.

AI Health Assistant: An AI assistant was integrated to answer health-related questions using user-specific data such as appointments and prescriptions. In this project, we plan to integrate a similar feature using API-based AI services within the dashboard.

Notifications System: Users received both in-app and external notifications for events such as messages, appointments, and pharmacy updates. This concept will be implemented in the web application through UI alerts and backend-triggered updates.

Admin Controls: Administrators were able to manage users and system data through full CRUD operations. This will be implemented in the Admin Panel page, where admins can create, update, and delete users and system records.

Overall, the 3090 system provides the core functionality and workflow design, while this project focuses on rebuilding it as a structured, component-based web application using React, APIs, and a database.

3 Problem Statement and Purpose

Healthcare systems often have poor communication between patients, doctors, and pharmacists. This makes scheduling appointments and managing prescriptions more difficult.

Our website solves this by providing a single system where users can book appointments, view prescriptions, and manage their information. The purpose of this project is to turn our static website into a working system where user actions update real data.

4 Goals and Objectives

Goals:

- Build a working web application with frontend, backend, and database
- Support different user roles (patients, doctors, admins)
- Connect the UI to real data using API requests

Objectives:

- Week 12: Set up React frontend structure, backend server, and database schema
- Week 13: Build core UI pages (Login, Dashboard, Appointments) and reusable components
- Week 14: Implement API routes (GET, POST, PUT, DELETE) for users, appointments, and prescriptions
- Week 15: Connect frontend to backend and implement role-based access (patient, doctor, admin)
- Week 16: Testing, debugging, and UI improvements

5 Project Description and System Design

The application is a multi-page web system where users can move between pages such as dashboard, appointments, and pharmacy.

Planned Pages and Functionality:

- **Home Page:** Landing page with general information about the hospital and navigation to login/signup.
- **Login / Signup Page:** Handles user authentication. Users can create accounts, log in, and be redirected based on their role.
- **Patient Dashboard:** Displays user-specific information such as upcoming appointments, prescriptions, notifications, and quick actions.
- **Appointments Page:** Allows patients to schedule appointments by selecting a date, time, and doctor. Users can also view, update, or cancel appointments. Doctors can confirm or reject requests.
- **Pharmacy Page:** Displays available medications and prescriptions. Users can view prescriptions, add items to a cart, and simulate checkout. Restricted medications are only visible if prescribed.
- **Doctor Portal:** Allows doctors to view patient appointments, confirm or reject bookings, and create prescriptions for patients.
- **Admin Panel:** Provides system-level control including managing users, viewing system data, and performing CRUD operations on users and medications.
- **Security and Settings Page:** Allows users to manage account security and personalization settings. Features include updating passwords, managing notification preferences, editing profile information, and viewing account activity. This page will help demonstrate secure user account management and role-based access control.
- **Support / Help Page:** Provides users with helpful information about how to use the platform, common support questions, contact information, and guidance for navigating hospital services. Unlike a simple static FAQ page, this page will serve as a user support resource integrated into the application.
- **Team Info / FAQ Page:** Displays project details and answers common user questions.

6 Component-Based Development

This project will follow a component-based approach using React.

Instead of repeating code across multiple pages like in our HTML version, we will build reusable components such as navigation bars, cards, forms, and dashboard sections. Each component will handle a specific part of the UI and can be reused across different pages.

By organizing the application this way, we can keep the code cleaner and easier to manage.

The application will use React components to update parts of the interface dynamically based on user actions and data from the backend.

6.1 Wireframes

The following wireframes show how the main pages will be structured.

6.1.1 Home Page / Dashboard

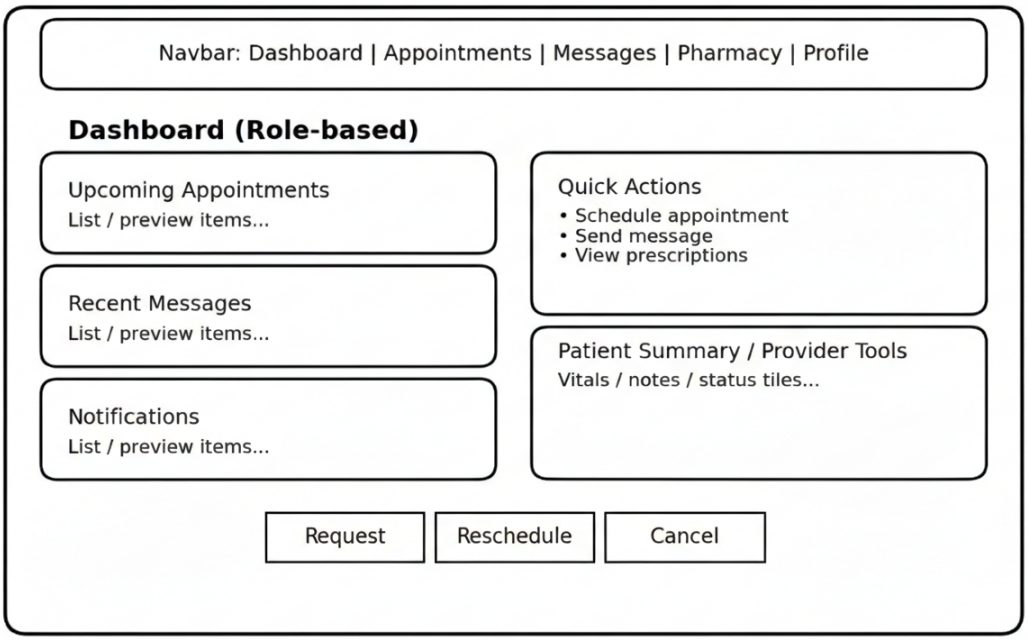


Figure 1: Dashboard layout showing appointments, messages, and actions

The dashboard shows important information such as appointments and notifications, along with quick actions.

6.1.2 Appointments Page

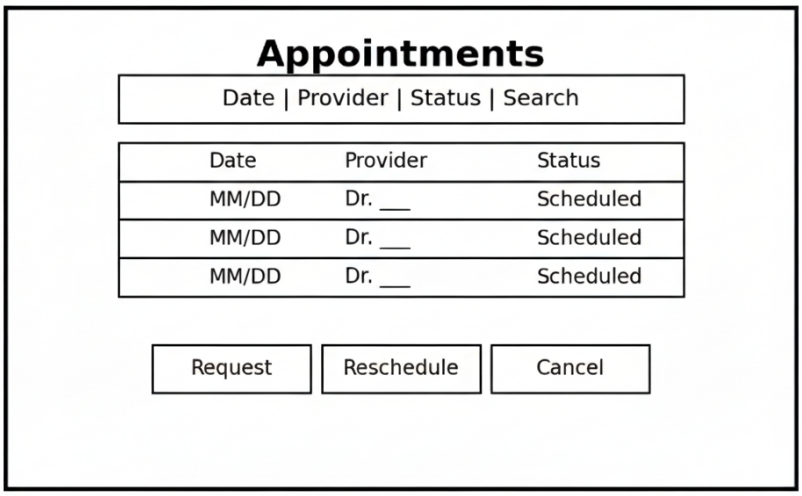


Figure 2: Appointments page with scheduling and management options

This page allows users to view, schedule, and manage appointments.

7 Feature Ownership and System Responsibilities

Devank Uppal Responsibilities:

- Frontend development for Login, Admin Panel, Pharmacy, and Team Info pages
- Backend APIs for user management and pharmacy system
- Database tables and queries for users and medications
- Implementation of authentication and role-based access control

Neal Sharma Responsibilities:

- Frontend development for Home, Dashboard, Appointments, and Doctor Portal pages
- Backend APIs for appointment scheduling and prescription system
- Database tables and queries for appointments and prescriptions

- Integration between frontend UI and backend services

Admins will have full access to create, update, and delete data, while regular users will have limited access to their own data.

8 API Design (High-Level)

The main data in our system includes users, appointments, and prescriptions.

Users:

- GET: Retrieve users
- POST: Create users
- PUT: Update user information
- DELETE: Remove users

Appointments:

- GET: View appointments
- POST: Book appointments
- PUT: Reschedule appointments
- DELETE: Cancel appointments

Prescriptions:

- GET: View prescriptions
- POST: Create prescriptions
- PUT: Update prescriptions
- DELETE: Remove prescriptions

9 Database Design

- Users Table: Stores account information and roles (patient, doctor, admin)
- Appointments Table: Stores appointment details including date, time, status, and user relationships
- Prescriptions Table: Stores medication data, dosage, and doctor-patient relationships
- Medications Table: Stores available pharmacy inventory

10 Technical Stack and Resources

- Frontend: React
- Backend: Node.js and Express
- Database: MySQL
- Tools: GitLab and Postman

11 GitLab Milestones and Planning

We will organize the project using GitLab milestones.

Milestone 1: Setup

- Set up frontend and backend
- Create project structure

Milestone 2: UI Development

- Build main pages and components

Milestone 3: Backend Integration

- Implement API routes
- Connect frontend to backend

Milestone 4: Feature Completion

- Finish appointments and prescriptions
- Implement user roles

Milestone 5: Testing

- Debug issues
- Improve UI and performance

12 Error Handling and Security

- Input validation on both frontend and backend
- Password hashing for user accounts
- Error pages for invalid routes or server issues

13 AI Feature (Optional Enhancement)

We plan to integrate an AI-based health assistant using Gemini.

This feature will allow users to ask basic health-related questions and receive general guidance. The AI will be connected through API calls and will work alongside the main system.

14 Final Comments

This project builds on our previous work and improves it into a more complete system. By connecting the frontend to a backend and database, we are making the application more realistic and useful.

Contact Information:

Devank Uppal: duppal@iastate.edu
Neal Sharma: nea1@iastate.edu

Devank Uppal Neal Sharma Spring 2026